

Tuyển tập Bài tập: Thuật toán Duyệt Đồ thị Cơ bản (BFS & DFS Problems)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Danh mục Bài tập Tuyển chọn

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
1	Building Roads	CSES 1666	● Cơ bản	Đếm thành phần liên thông bằng DFS/BFS	$O(V + E)$
2	Message Route	CSES 1667	● Cơ bản	BFS đường đi ngắn nhất không trọng số & truy vết	$O(V + E)$
3	Counting Rooms	CSES 1192	● Cơ bản	Flood Fill loang vùng trên lưới bằng BFS/DFS	$O(N \cdot M)$
4	Building Teams	CSES 1668	● Cơ bản	Kiểm tra đồ thị hai phía (2-Coloring Bipartite)	$O(V + E)$
5	Round Trip II	CSES 1678	● Cơ bản	Phát hiện chu trình có hướng bằng DFS 3 màu	$O(V + E)$
6	Round Trip	CSES 1669	● Trung cấp	Phát hiện chu trình vô hướng bằng DFS & in chu trình	$O(V + E)$
7	Course Schedule	CSES 1679	● Trung cấp	Sắp xếp tô-pô bằng thuật toán Kahn (BFS in-degree)	$O(V + E)$
8	Labyrinth	CSES 1193	● Trung cấp	BFS mê cung 2D truy vết chuỗi bước đi (L, R, U, D)	$O(N \cdot M)$
9	Monsters	CSES 1194	● Trung cấp	Multi-source BFS lan tỏa quái vật song song	$O(N \cdot M)$
10	Wizard in Maze	AtCoder ABC 176D	● Trung cấp	0-1 BFS với <code>std::deque</code> (đi bộ 0, biến hình 1)	$O(N \cdot M)$

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
11	Labyrinth CF	Codeforces 1063B	● Trung cấp	0-1 BFS tối thiểu số bước di chuyển sang trái/phải	$O(N \cdot M)$
12	Fox and Names	Codeforces 510C	● Trung cấp	Xây dựng đồ thị có hướng từ thứ tự từ điển & Topo	$O(N \cdot L + \Sigma)$
13	Planets Cycles	CSES 1751	● Nâng cao	Cấu trúc Đồ thị Hàm (Functional Graph Rho-shape)	$O(N)$
14	Planets Queries I	CSES 1750	● Nâng cao	Functional Graph kết hợp Binary Lifting nhảy K bước	$O(N \log K + Q \log K)$
15	Tree Distances I	CSES 1132	● Nâng cao	Hai lần BFS tìm đường kính cây và khoảng cách xa nhất	$O(N)$
16	Flight Routes Check	CSES 1682	● Nâng cao	Kiểm tra liên thông mạnh 2 lượt BFS (thuật toán Kosaraju)	$O(V + E)$

● Phần 1: Cấp độ Cơ bản & Khởi động (Warm-up)

Bài 1: Building Roads (CSES 1666)

- **Đề bài:** Vương quốc Byteland có N thành phố và M con đường hai chiều nối giữa các thành phố. Hãy tìm số lượng con đường mới tối thiểu cần xây dựng sao cho mọi thành phố đều có thể đi lại được với nhau, và chỉ ra danh sách các con đường đó.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
 - *Hint 1:* Bài toán yêu cầu kết nối các thành phần liên thông rời rạc của đồ thị vô hướng.
 - *Hint 2:* Sử dụng DFS hoặc BFS để tìm tất cả các thành phần liên thông. Mỗi thành phần chọn ra một đỉnh đại diện (ví dụ đỉnh đầu tiên được duyệt).
 - *Hint 3:* Nếu có K thành phần liên thông, số đường cần thêm là $K - 1$. Chỉ cần nối đỉnh đại diện của thành phần 1 với đỉnh đại diện của các thành phần $2, 3, \dots, K$.

Bài 2: Message Route (CSES 1667)

- **Đề bài:** Mạng máy tính gồm N máy tính và M kết nối mạng hai chiều. Mai muốn gửi tin nhắn từ máy 1 tới máy N qua ít máy trung gian nhất. Hãy tìm số lượng máy tính trên lộ trình ngắn nhất và in ra lộ trình đó. Nếu không thể gửi tin nhắn, in `IMPOSSIBLE`.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
 - *Hint 1:* Vì đồ thị không có trọng số (mỗi kết nối có chi phí là 1), thuật toán BFS là lựa chọn tối ưu để tìm đường đi ngắn nhất trong $O(V + E)$.

- *Hint 2:* Khởi tạo `dist[1] = 0`, đẩy 1 vào hàng đợi. Khi duyệt đỉnh kề v chưa thăm của u , gán `dist[v] = dist[u] + 1`, `parent[v] = u`, và đẩy v vào queue.
- *Hint 3:* Sau BFS, nếu `dist[N] == INF`, in `IMPOSSIBLE`. Ngược lại, truy vết ngược từ N về 1 bằng mảng `parent` rồi đảo ngược lại.

Bài 3: Counting Rooms (CSES 1192)

- **Đề bài:** Cho bản đồ một tòa nhà kích thước $N \times M$ gồm các ô sàn `.` và tường `#`. Một căn phòng là một vùng các ô sàn liên thông theo 4 hướng (trên, dưới, trái, phải). Hãy đếm tổng số căn phòng trong tòa nhà.
- **Ràng buộc:** $1 \leq N, M \leq 1000$.
- **Gợi ý:**
- *Hint 1:* Đây là bài toán loang vùng (Flood Fill) kinh điển trên lưới 2D.
- *Hint 2:* Duyệt qua từng ô (r, c) của bản đồ: Nếu ô đó là `.` và chưa được thăm, tăng biến đếm số phòng lên 1 và kích hoạt hàm `dfs(r, c)` hoặc `bfs(r, c)` để đánh dấu toàn bộ các ô sàn thuộc cùng phòng đó.
- *Hint 3:* Dùng mảng 4 hướng: `dr = {-1, 1, 0, 0}`, `dc = {0, 0, -1, 1}`. Độ phức tạp $O(N \cdot M)$.

Bài 4: Building Teams (CSES 1668)

- **Đề bài:** Có N học sinh và M cặp tình bạn. Cần chia học sinh vào 2 nhóm sao cho không có hai người bạn nào ở cùng một nhóm. Nếu có thể, hãy in ra nhóm của từng người (1 hoặc 2); nếu không thể, in `IMPOSSIBLE`.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Quy đổi bài toán về kiểm tra **Đồ thị hai phía (Bipartite Graph Check)** và tô màu bằng 2 màu (1 và 2).
- *Hint 2:* Duyệt qua tất cả các đỉnh từ 1 đến N . Nếu đỉnh chưa tô màu, gán màu 1 và chạy BFS/DFS: các đỉnh kề sẽ nhận màu đối nghịch (`3 - color[u]`).
- *Hint 3:* Nếu gặp một đỉnh kề đã có màu và màu đó trùng với đỉnh hiện tại, đồ thị chứa chu trình lẻ $\implies$ in `IMPOSSIBLE`.

Bài 5: Round Trip II (CSES 1678)

- **Đề bài:** Byteland có N thành phố và M chuyến bay một chiều. Hãy tìm một lộ trình bay khép kín (chu trình có hướng) bắt đầu và kết thúc tại cùng một thành phố, đi qua ít nhất 2 thành phố khác. Nếu có nhiều chu trình, in ra chu trình bất kỳ; nếu không có, in `IMPOSSIBLE`.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Phát hiện chu trình trên **đồ thị có hướng** bắt buộc phải dùng kỹ thuật DFS 3 màu: Trắng (0), Xám (1), Đen (2).

- *Hint 2:* Khi đang ở đỉnh u (màu Xám), nếu duyệt tới đỉnh kề v cũng có màu Xám, ta đã tìm thấy một Cạnh ngược (Back edge) $u \rightarrow v \implies$ Tồn tại chu trình!
- *Hint 3:* Lưu vết đường đi qua mảng `parent`. Khi phát hiện chu trình tại $u \rightarrow v$, truy vết từ u ngược về v qua `parent`, sau đó thêm v vào cuối để khép kín chu trình.

● Phần 2: Cấp độ Trung cấp — Kỹ thuật Đồ thị Cốt lõi

Bài 6: Round Trip (CSES 1669)

- **Đề bài:** Tương tự Round Trip II nhưng trên **đồ thị vô hướng**. Tìm một chu trình có ít nhất 3 thành phố bắt đầu và kết thúc tại cùng một đỉnh.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Trên đồ thị vô hướng, cạnh ngược về đỉnh cha trực tiếp `p` không được tính là chu trình. Do đó trong hàm `dfs(u, p)`, nếu $v == p$ thì bỏ qua.
- *Hint 2:* Nếu $v \neq p$ và v đã được thăm (`visited[v] == true`), ta đã phát hiện chu trình khép kín giữa u và v .
- *Hint 3:* Dừng DFS ngay khi tìm thấy chu trình đầu tiên và truy vết từ u về v , in kết quả.

Bài 7: Course Schedule (CSES 1679)

- **Đề bài:** Có N khóa học cần hoàn thành. Một số khóa học có điều kiện tiên quyết: khóa a phải học trước khóa b . Hãy tìm một thứ tự hoàn thành tất cả N khóa học hợp lệ, hoặc in `IMPOSSIBLE` nếu tồn tại ràng buộc luẩn quẩn.
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Đây là bài toán **Sắp xếp Tô-pô (Topological Sort)** trên Đồ thị có hướng không chu trình (DAG).
- *Hint 2:* Sử dụng thuật toán Kahn: Tính bán bậc vào `in_degree[u]`. Nạp tất cả các đỉnh có `in_degree == 0` vào hàng đợi.
- *Hint 3:* Mỗi khi lấy u ra khỏi hàng đợi, thêm u vào danh sách kết quả và giảm bán bậc vào của các đỉnh kề. Nếu danh sách kết quả có độ dài $< N$, đồ thị có chu trình $\implies$ in `IMPOSSIBLE`.

Bài 8: Labyrinth (CSES 1193)

- **Đề bài:** Cho mê cung $N \times M$ gồm các ô trống `.`, tường `#`, điểm xuất phát `A` và đích `B`. Hãy tìm đường đi ngắn nhất từ `A` đến `B` và in ra chuỗi các bước di chuyển tương ứng (`U` - Lên, `D` - Xuống, `L` - Trái, `R` - Phải).
- **Ràng buộc:** $1 \leq N, M \leq 1000$.
- **Gợi ý:**
- *Hint 1:* Chạy BFS từ ô `A`. Lưu ma trận `parent_dir[r][c]` ghi nhận hướng di chuyển đã dẫn tới ô (r, c) (ví dụ: 'U', 'D', 'L', 'R').
- *Hint 2:* Lưu ma trận `parent_cell[r][c]` để biết tọa độ ô trước đó.

- *Hint 3:* Khi chạm tới `B`, lần theo các bước đi ngược về `A`, lưu các ký tự hướng đi vào xâu, sau đó đảo ngược xâu và in ra.

Bài 9: Monsters (CSES 1194)

- **Đề bài:** Bạn đang ở ô `A` trong mê cung và muốn thoát ra một ô biên bất kỳ. Tuy nhiên trong mê cung có nhiều con quái vật `M`. Mỗi giây, bạn và tất cả quái vật đều có thể di chuyển 1 ô theo 4 hướng. Quái vật luôn chọn đường ngắn nhất tới bạn. Bạn có thể thoát hiểm thành công không?
- **Ràng buộc:** $1 \leq N, M \leq 1000$.
- **Gợi ý:**
- *Hint 1:* Sử dụng kỹ thuật **Multi-Source BFS** cho quái vật trước: Đẩy toàn bộ các vị trí có quái vật vào hàng đợi ban đầu và tính ma trận thời gian quái vật có thể tới sớm nhất: `monster_time[r][c]`.
- *Hint 2:* Chạy lượt BFS thứ hai xuất phát từ người (`A`), tính thời gian người tới ô (r, c) là `human_time[r][c]`.
- *Hint 3:* Người chỉ có thể bước vào ô (r, c) nếu `human_time[r][c] < monster_time[r][c]`. Nếu người chạm tới một ô ở viền ngoài mê cung, ta truy vết và in đường đi thoát hiểm!

Bài 10: Wizard in Maze (AtCoder ABC 176D)

- **Đề bài:** Pháp sư ở ô (C_h, C_w) muốn tới (D_h, D_w) trên mê cung $H \times W$. Có 2 cách di chuyển: Đi bộ sang 4 ô kề cạnh (chi phí 0) hoặc dùng phép dịch chuyển ma thuật tới bất kỳ ô nào trong phạm vi hình vuông 5×5 quanh mình (chi phí 1). Tìm chi phí ma thuật tối thiểu.
- **Ràng buộc:** $1 \leq H, W \leq 1000$.
- **Gợi ý:**
- *Hint 1:* Trọng số di chuyển chỉ nhận hai giá trị 0 và 1. Đây là bài toán kinh điển của **0-1 BFS** với `std::deque`.
- *Hint 2:* Với mỗi ô (r, c) , duyệt 4 ô kề: nếu hợp lệ và chi phí mới $< dist$, cập nhật và `push_front`.
- *Hint 3:* Duyệt các ô trong phạm vi 5×5 : nếu hợp lệ và chi phí $+1 < dist$, cập nhật và `push_back`. Độ phức tạp $O(H \cdot W)$.

Bài 11: Labyrinth (Codeforces 1063B)

- **Đề bài:** Cho mê cung $N \times M$, điểm xuất phát (r, c) , và hai số nguyên X, Y là số bước tối đa được phép rẽ sang trái và sang phải. Số bước đi lên và đi xuống không bị giới hạn. Hỏi có thể tiếp cận được bao nhiêu ô trong mê cung?
- **Ràng buộc:** $1 \leq N, M \leq 2000, 0 \leq X, Y \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Nhận xét: Đi lên và đi xuống không tiêu tốn tài nguyên rẽ trái/phải. Đi sang trái tốn 1 bước trái, đi sang phải tốn 1 bước phải.
- *Hint 2:* Dùng **0-1 BFS** với trạng thái tối thiểu hóa số lần rẽ trái (hoặc rẽ phải). Đi lên/xuống có trọng số 0 (`push_front`), đi sang trái/phải có trọng số 1 (`push_back`).

- *Hint 3:* Sau khi tìm được số lần rẽ trái/phải tối thiểu tới mỗi ô, kiểm tra xem có thỏa mãn $\leq X$ và $\leq Y$ hay không để đếm tổng số ô.

Bài 12: Fox and Names (Codeforces 510C)

- **Đề bài:** Cáo Ciel muốn sắp xếp lại thứ tự bảng chữ cái tiếng Anh gồm 26 chữ cái sao cho một danh sách N tên cho trước xuất hiện theo đúng thứ tự từ điển mới này. Hãy tìm một thứ tự bảng chữ cái hợp lệ hoặc in `Impossible`.
- **Ràng buộc:** $1 \leq N \leq 100$, độ dài mỗi tên ≤ 100 .
- **Gợi ý:**
- *Hint 1:* So sánh hai từ liên tiếp S_i và S_{i+1} trong danh sách. Tìm ký tự khác nhau đầu tiên tại vị trí k : ký tự $S_i[k]$ bắt buộc phải đứng trước $S_{i+1}[k]$ trong bảng chữ cái mới $\implies$ Thêm cạnh có hướng $S_i[k] \rightarrow S_{i+1}[k]$.
- *Hint 2:* Nếu S_{i+1} là tiền tố ngắn của S_i (ví dụ: `apple` đứng trước `app`), bài toán vô nghiệm ngay lập tức.
- *Hint 3:* Chạy thuật toán Kahn hoặc DFS Topological Sort trên đồ thị 26 chữ cái. Nếu có chu trình, in `Impossible`.

● Phần 3: Cấp độ Nâng cao — ICPC & Thi Đấu Đỉnh cao

Bài 13: Planets Cycles (CSES 1751)

- **Đề bài:** Có N hành tinh. Mỗi hành tinh có một cổng viễn dịch dẫn tới duy nhất một hành tinh khác t_i . Với mỗi hành tinh, hãy tính số lượng hành tinh khác nhau bạn sẽ ghé thăm nếu liên tục dịch chuyển qua các cổng.
- **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5$, $1 \leq t_i \leq N$.
- **Gợi ý:**
- *Hint 1:* Đây là **Đồ thị Hàm (Functional Graph)** với $\text{out-degree}(u) = 1$. Mỗi thành phần liên thông có cấu trúc hình chữ Rho: gồm đúng 1 chu trình và các nhánh cây đổ vào chu trình.
- *Hint 2:* Dùng DFS với 3 trạng thái hoặc thuật toán Kahn trên đồ thị ngược để tìm tất cả các đỉnh nằm trong chu trình. Tất cả các đỉnh thuộc cùng một chu trình độ dài L đều có đáp án là L .
- *Hint 3:* Với các đỉnh nằm trên nhánh cây dẫn vào chu trình, đáp án bằng khoảng cách từ đỉnh đó tới chu trình cộng với độ dài chu trình (sử dụng quy hoạch động trên cây con). Độ phức tạp $O(N)$.

Bài 14: Planets Queries I (CSES 1750)

- **Đề bài:** Cho đồ thị hàm N đỉnh với cổng dịch chuyển t_i . Có Q truy vấn, mỗi truy vấn gồm (x, k) : Hỏi xuất phát từ hành tinh x và dịch chuyển đúng k bước thì sẽ dừng lại ở hành tinh nào?
- **Ràng buộc:** $1 \leq N, Q \leq 2 \cdot 10^5$, $1 \leq k \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Với $k \leq 10^9$, ta không thể mô phỏng từng bước. Áp dụng kỹ thuật **Binary Lifting (Nhảy nhị phân)** trên đồ thị hàm.

- *Hint 2:* Xây dựng bảng `up[node][j]` là hành tinh đến được sau 2^j bước:

$$up[u][0] = t_u, \quad up[u][j] = up[up[u][j-1]][j-1]$$

- *Hint 3:* Với mỗi truy vấn (x, k) , duyệt các bit của k : nếu bit thứ j bật, nhảy $x \leftarrow up[x][j]$. Tiền xử lý $O(N \log K)$, mỗi truy vấn $O(\log K)$.

Bài 15: Tree Distances I (CSES 1132)

- **Đề bài:** Cho một cây gồm N nút. Với mỗi nút $u \in [1, N]$, hãy tìm khoảng cách lớn nhất từ nút u tới một nút bất kỳ khác trên cây.
- **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Định lý Cây: Nút xa nhất của bất kỳ nút nào trên cây luôn là một trong hai đầu mút của **Đường kính cây (Tree Diameter)**!
- *Hint 2:* Chạy BFS lần 1 từ nút 1 tìm nút A xa nhất. Chạy BFS lần 2 từ A tìm nút B xa nhất $\implies$ đoạn (A, B) là đường kính của cây.
- *Hint 3:* Chạy BFS từ A để tính khoảng cách `distA[u]` và BFS từ B để tính `distB[u]`. Đáp án cho nút u là $\max(distA[u], distB[u])$. Tổng thời gian $O(N)$ bằng đúng 3 lần BFS!

Bài 16: Flight Routes Check (CSES 1682)

- **Đề bài:** Có N thành phố và M chuyến bay một chiều. Hãy kiểm tra xem từ bất kỳ thành phố nào có thể bay tới được mọi thành phố khác hay không. Nếu không, hãy chỉ ra một cặp (u, v) sao cho không thể đi từ u tới v .
- **Ràng buộc:** $1 \leq N \leq 10^5, 1 \leq M \leq 2 \cdot 10^5$.
- **Gợi ý:**
- *Hint 1:* Đồ thị có thể đi lại giữa mọi cặp đỉnh khi và chỉ khi đồ thị là **Liên thông mạnh (Strongly Connected)**.
- *Hint 2:* Theo bổ đề Kosaraju, ta chỉ cần kiểm tra từ đỉnh 1:
 - Chạy BFS từ đỉnh 1 trên đồ thị gốc G : nếu tồn tại đỉnh v không thăm được $\implies$ in cặp $(1, v)$.
 - Xây dựng đồ thị chuyển vị G^{rev} (đảo ngược hướng tất cả các cạnh) và chạy BFS từ đỉnh 1: nếu tồn tại đỉnh u không thăm được $\implies$ trong đồ thị gốc không có đường từ u tới 1 $\implies$ in cặp $(u, 1)$.
- *Hint 3:* Nếu cả hai lần BFS đều thăm được hết N đỉnh, đồ thị liên thông mạnh $\implies$ in `YES`. Độ phức tạp $O(V + E)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.